Übungsblatt 3

Web-Anwendungen und Web-Architekturen

5430UE Web and Data Engineering

29. April 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Unterscheiden von Kategorien von Web-Anwendungen und Zuordnung typischer Systeme
- Zentrale Prinzipien des Web Engineerings (z. B. SoC, SRP, DRY) verstehen, erkennen und anwenden
- Analyse grundlegender Architektur- und Betriebsaspekte von Web-Anwendungen

Kategorien von Web-Anwendungen

Übung 3.1: Kategorien von Web-Anwendungen

Ordnen Sie folgende Systeme einer Web-Anwendungskategorie zu (statisch, dynamisch server-seitig, Single-Page Application, Web-Service/API). Begründen Sie eine der Zuordnungen kurz.

System	Kategorie
Universitäts-Raumbelegungsplan als HTML-Seite, wöchentlich neu generiert	?
Online-Banking-Portal mit Echtzeit-Kontoübersicht	?
Wetter-REST-API, die JSON zurückgibt	?
Produktkatalog mit Suchfilter ohne Neuladen der Seite	?

Web Engineering — Ziele und Prinzipien



Übung 3.2: Web Engineering — Ziele und Prinzipien

Web Engineering ist die systematische Disziplin zur Entwicklung von Web-Anwendungen.

1. Geben Sie die Grundprinzipien des Web Engineering in eigenen Worten an.
2. Formulieren Sie in eigenen Worten: Was unterscheidet Web Engineering von klassischer Software-Entwicklung?
3. Nennen Sie drei typische Qualitätsziele, die Web-Anwendungen im Gegensatz zu Desktop-Software besonders stark betonen.
4. Ein Unternehmen entwickelt seine Web-Anwendung ohne Versionskontrolle, ohne Tests und ohne dokumentierte Anforderungen. Welche konkreten Probleme entstehen kurz- und langfristig?

Ressourcen-Engpässe identifizieren

Übung 3.3: Ressourcen-Engpässe identifizieren

Ein Online-Shop beobachtet die in der Tabelle aufgeführten Symptome. Geben Sie jeweils an, bei welcher Ressource - Rechenleistung (CPU), Arbeitsspeicher (RAM), persistenter Speicher (Storage), Netzwerk (I/O) - vermutlich der Engpass vorliegt, der zum jeweiligen Symptom führt. Begründen Sie jede Zuordnung in einem Satz.

Symptom	Vermuteter Engpass
Antwortzeiten steigen bei komplexen Preisberechnungen	?
Server stürzt ab wenn 50.000 Sessions gleichzeitig aktiv sind	?
Produktbilder laden sehr langsam	?
API-Requests von mobilen Clients laufen regelmäßig in Timeouts	?

Vertikale vs. Horizontale Skalierung

Übung 3.4: Vertikale vs. Horizontale Skalierung

Wir betrachten folgendes Szenario: *Ein Start-up betreibt seinen Web-Shop auf einem einzelnen Server (8 Kerne, 32 GB RAM). Mit großer werdender Popularität des Start-ups wächst der Traffic um Faktor 10.*

1. Beschreiben Sie eine **vertikale** Skalierungsmaßnahme und nennen Sie ihre Grenze.
2. Beschreiben Sie eine **horizontale** Skalierungsmaßnahme und die dabei entstehenden neuen Herausforderungen.
3. Warum ist Session-State ein kritisches Problem bei horizontaler Skalierung, und wie löst man es?

Schichtenzuordnung

Übung 3.5: Schichtenzuordnung

Ein Entwickler implementiert eine Bestellfunktion.

```
// (A)
<form action="/order" method="POST">
   <input name="productId" type="hidden" value="42"/>
   <button type="submit">Jetzt kaufen</button>
</form>

// (B)
if (product.getStock() < quantity)
  throw new InsufficientStockException();
double total = product.getPrice() * quantity * taxService.getRate();
emailService.sendConfirmation(user, order);

// (C)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findById(@Param("id") Long id);
```

Ordnen Sie die Code-Fragmente den drei Schichten des 3-Schichten-Modells zu (Präsentation / Anwendung / Persistenz) und begründen Sie Ihre Zuordnung kurz.

Architekturprinzipien anwenden

Übung 3.6: Architekturprinzipien anwenden

Im folgenden Code sind die Architekturprinzipien Separation of Concerns (SoC), Single Responsibility (SRP) und Don't Repeat Yourself (DRY) verletzt:

```
@PostMapping("/checkout")
public String checkout(HttpServletRequest req, Model model) {
  String userId = req.getSession().getAttribute("userId").toString();
  ResultSet rs = db.query("SELECT * FROM cart WHERE user_id = " + userId);
  double total = 0;
  while (rs.next()) {
    total += rs.getDouble("price") * rs.getInt("qty") * 1.19;
  }
  // gleiche Steuerberechnung wie in /invoice und /quote
  String html = "<h1>Total: Euro" + total + "</h1>";
  model.addAttribute("content", html);
  sendEmail(userId, "Bestellung eingegangen, Total: Euro" + total);
  return "result";
}
```

1. Welche Prinzipien werden verletzt? Beschreiben Sie je eine konkrete Verletzung.

2. Skizzieren Sie eine verbesserte Struktur (Angabe der Klassennamen genügt).



Git – Merge, Konfliktauflösung und .gitignore

Übung 3.7: Git (1/3)

Verwenden Sie für diese Aufgabe das Repository aus Übung 2, Aufgabe 2, das Sie bereits lokal geklont haben. Führen Sie vor Beginn der Aufgabe einen Pull durch, um sicherzugehen, dass das lokale Repository aktuell ist.

Führen Sie alle folgenden Schritte in Ihrem lokalen Repository aus.

1. Arbeiten mit Branches

- Erstellen Sie im `main`-Branch eine neue Datei `notes.txt` mit Inhalt *Hallo Welt!*
- Fügen Sie die Datei zur Versionskontrolle hinzu und committen Sie diese.
- Erstellen Sie ausgehend vom aktuellen `main`-Branch einen neuen Branch `feature/update-text` und wechseln Sie auf diesen.
- Bearbeiten Sie nun die Datei `notes.txt` in beiden Branches unterschiedlich:
  - Im Branch `feature/update-text`: Ändern Sie den Inhalt zu *Version B* und committen Sie die Änderung.
  - Wechseln Sie zurück auf den Branch `main`: Ändern Sie den Inhalt zu *Version A* und committen Sie die Änderung.

Übung 3.7: Git (1/3)

Merge-Konflikt

1. Führen Sie einen Merge des Branches `feature/update-text` in den `main`-Branch durch.
2. Beschreiben Sie, wie Git einen Merge-Konflikt in der Datei darstellt.
3. Lösen Sie den Merge-Konflikt so, dass die finale Datei Inhalt *Version A & Version B* hat.
4. Geben Sie die notwendigen Git-Befehle an, um die Konfliktlösung abzuschließen.

Übung 3.7: Git (2/3)

2. .gitignore

In vielen Projekten sollen bestimmte Dateien nicht versioniert werden (z. B. temporäre Dateien oder Logs).

1. Erklären Sie kurz den Zweck der Datei `.gitignore`.
2. Erstellen Sie eine `.gitignore`-Datei mit folgenden Regeln:
   - Ignoriere alle Dateien mit der Endung `.log`
   - Ignoriere den Ordner `temp/`
   - Ignoriere die Datei `config.local.json`
3. Warum werden bereits getrackte Dateien nicht automatisch durch `.gitignore` ignoriert?

Übung 3.7: Git (3/3)

3. Zustandsanalyse eines Repositories

Gegeben sei folgendes Szenario in einem lokalen Repository. Die Datei `.gitignore` aus der vorherigen Teilaufgabe ist bereits vorhanden. Ausgangspunkt ist ein Commit mit folgendem Inhalt der Datei `data.txt`:

```
Hello World
-
-
Hello WDE!
```

Es werden folgende Befehle ausgeführt:

```
git branch feature
git checkout feature
```

(Fortsetzung auf der nächsten Folie)

Übung 3.7: Git - Zustandsanalyse (Fortsetzung)

Im Branch `feature` wird die zweite Zeile geändert und committet:

```
Hello World
-
-
Hello from Feature
```

```
git add data.txt
git commit -m "Feature change"
git checkout main
```

Im Branch `main` wird die erste Zeile geändert und committet:

```
Hello from Main
-
-
Hello WDE!
```

(Fortsetzung auf der nächsten Folie)



Übung 3.7: Git - Zustandsanalyse (Fortsetzung 2)

```
git add data.txt
git commit -m "Main change"
echo "Debug info" > debug.log
git merge feature
```

1. Welchen Inhalt hat die Datei `data.txt` im `main`-Branch nach dem Merge?
2. Wird die Datei `debug.log` von Git verfolgt? Begründen Sie Ihre Antwort.
3. Wie sieht der Stand des Remote-Repositories (`origin/main`) nach diesem Ablauf aus? Begründen Sie Ihre Antwort.

JavaScript



Übung 3.8: JavaScript

Hinweis: Diese Aufgabe wird in der Übung nicht explizit behandelt. Sie dient der eigenständigen Wiederholung und Festigung der JavaScript-Kenntnisse sowie als Vorbereitung auf spätere Aufgaben und das Projekt.

Bearbeiten Sie die kostenlosen (ersten) Kapitel von learnjavascript.online. Alternativ können Sie auch den umfangreichen kostenlosen JavaScript Basiskurs von freeCodeCamp nutzen.

Eine gute Möglichkeit einzelne Aspekte von JavaScript nachzuschlagen ist w3school.com.